

CAPACITOR

Capacitor est un framework **cross-platform** développé par l'équipe Ionic, permettant de transformer une application web Angular (mais aussi React ou Vue.js) en **application mobile native** pour Android et iOS. Capacitor offre un **pont natif performant** avec accès direct aux APIs mobiles (caméra, GPS, notifications, etc.) tout en conservant votre code web existant. Cette approche **hybride moderne** permet un développement rapide avec un seul codebase pour iOS, Android et Web.

Prérequis

Avant de commencer l'intégration de Capacitor dans un projet Angular, il est **nécessaire de disposer** :

- d'un projet Angular déjà créé ;
- de Node.js et npm installés ;
- d'un environnement de développement configuré ;
- IntelliJ IDEA ou Android Studio pour la partie Android.

Installation de Capacitor

Depuis le **terminal** du projet Angular, exécuter la commande suivante :

```
ng add @capacitor/angular
```

Lors de l'installation, il est demandé de renseigner un identifiant d'application appelé **Package ID** ou **Bundle ID**. Celui-ci doit respecter une notation inversée de type nom de domaine.

Exemples :

```
com.nomentreprise.app
```

```
com.nomprojet.mobile
```

Génération du build Angular

Une fois Capacitor ajouté au projet, il faut générer **une version de production** de l'application Angular :

```
ng build --configuration production
```

Cette commande produit les fichiers compilés du projet qui seront utilisés par Capacitor.

Configuration du fichier Capacitor

Dans le fichier `capacitor.config.ts`, il faut vérifier et modifier la propriété `webDir` afin qu'elle pointe vers le dossier de build Angular.

Exemple :

```
webDir: 'dist/nom-projet/browser'
```

Cette configuration permet à Capacitor de savoir où se trouvent les fichiers de l'application web à embarquer dans l'application mobile.

Installation des plateformes natives

Pour ajouter la prise en charge des plateformes mobiles, **installer** les **dépendances** nécessaires :

```
npm i @capacitor/ios @capacitor/android
```

Ensuite, ajouter la plateforme Android au projet :

```
npx cap add android
```

Configuration de l'environnement Android

Pour exécuter l'application sur Android, il est nécessaire de configurer l'environnement SDK dans IntelliJ IDEA ou Android Studio.

Étapes de configuration

1. Ouvrir IntelliJ IDEA.
2. Aller dans **Settings > Plugins**.
3. Installer le plugin **Android**.
4. Aller dans **Languages & Frameworks > Android SDK**.

5. Dans l'onglet **SDK Platforms**, installer **Android 16.0 (API level 36)**.

6. Dans l'onglet **SDK Tools**, installer :

- Android SDK Build-Tools ;
- Android SDK Command-line Tools ;
- Android Emulator ;
- Android SDK Platform-Tools.

7. Copier le chemin indiqué dans **Android SDK Location**.

Création du fichier **local.properties**

Dans le dossier **android** du projet, créer un fichier **local.properties**, puis renseigner le chemin du SDK Android :

sdk.dir=chemin/vers/Android/Sdk

Ce fichier permet au projet Android de localiser correctement le SDK installé sur la machine.

Ajout d'Ionic au projet

Si l'interface mobile doit bénéficier de composants Ionic, il est possible d'ajouter Ionic au projet Angular avec la commande suivante :

ng add @ionic/angular

Cette étape est optionnelle, mais elle peut être utile pour améliorer l'ergonomie et l'apparence de l'application mobile.

Synchronisation et lancement Android

Avant chaque exécution, il est recommandé de reconstruire le projet Angular puis de synchroniser les fichiers avec Capacitor.

Commandes à exécuter :

ng build --configuration production génère la version de production du projet Angular ;

npx cap sync synchronise les fichiers web avec les plateformes natives ;

npx cap run android lance l'application sur Android.

Utilisation d'un émulateur Android

Pour tester l'application sur un appareil virtuel :

1. Ouvrir IntelliJ IDEA ou Android Studio.
2. Ouvrir le dossier **android** du projet.
3. Accéder au gestionnaire d'appareils virtuels.
4. Sélectionner un téléphone de type **Medium Phone**.
5. Démarrer l'émulateur.
6. Lancer ensuite l'application Android depuis le projet.